



Concurso Internacional de Programação Colegiada

2023

Concursos regionais da América Latina

21 de outubro de 2023

Sessão do concurso

Este conjunto de problemas contém 13 problemas; as páginas são numeradas de 1 a 21.

Esse conjunto de problemas é usado em competições simultâneas com os seguintes países participantes:

Antígua e Barbuda, Argentina, Bolívia, Brasil, Chile, Colômbia, Costa Rica, Cuba, El Salvador, Guatemala, México, Porto Rico, República Dominicana, Trinidad e Tobago, Uruguai e Venezuela

Informações gerais

Salvo indicação em contrário, as condições a seguir são válidas para todos os problemas.

Nome do programa

1. Sua solução deve se chamar `codename.c`, `codename.cpp`, `codename.java`, `codename.kt`, `codename.py3`, em que `codename` é a letra maiúscula que identifica o problema.

Entrada

1. A entrada deve ser lida a partir da entrada padrão.
2. A entrada consiste em um único caso de teste, que é descrito usando um número de linhas que depende do problema. Nenhum dado extra aparece na entrada.
3. Quando uma linha de dados contém vários valores, eles são separados por espaços *simples*. Nenhum outro espaço aparece na entrada. Não há linhas vazias.
4. É usado o alfabeto inglês. Não há letras com tildes, acentos, diaereses ou outros sinais diacríticos (`ñ`, `Ã`, `'e`, `İ`, `^o`, `Ü`, `ç`, etc.).
5. Cada linha, inclusive a última, tem a marca de fim de linha usual.

Saída

1. A saída deve ser gravada na saída padrão.
2. O resultado do caso de teste deve aparecer na saída usando um número de linhas que depende do problema. Nenhum dado extra deve aparecer na saída.
3. Quando uma linha de resultados contém vários valores, eles devem ser separados por espaços *simples*. Nenhum outro espaço deve aparecer na saída. Não deve haver linhas vazias.
4. O alfabeto inglês deve ser usado. Não deve haver letras com tildes, acentos, diaereses ou outros sinais diacríticos (`ñ`, `Ã`, `'e`, `İ`, `^o`, `Ü`, `ç`, etc.).
5. Cada linha, inclusive a última, deve ter a marca de fim de linha usual.
6. Para gerar números reais, arredonde-os para o racional mais próximo com o número necessário de dígitos após o ponto decimal. O caso de teste é tal que não há empates ao arredondar conforme especificado.

Problema A -Análise de contratos

Autor : Agust'in Santiago Gutierrez, Argentina

O Dr. Kruskal está iniciando um negócio de comércio de tiberium. Eles têm N possíveis fornecedores de tiberium e muitos clientes interessados em receber tiberium para administrar suas próprias indústrias.

Os dias do calendário são numerados cronologicamente usando números inteiros positivos, e cada fornecedor é identificado por um número inteiro distinto de 1 a N . O fornecedor i pode fornecer tiberium em qualquer dia a partir do dia S_i , mas não nos dias estritamente anteriores a S_i . Ele cobra um preço de P_i dólares por dia por esse serviço. Como Kruskal é muito inteligente, a lista de fornecedores contém apenas os melhores fornecedores da cidade. Além disso, é o caso de $S_i < S_{i+1}$ e $P_i > P_{i+1}$ para $i = 1, 2, \dots, N - 1$. O sistema da Kruskal mantém um banco de dados de clientes disponíveis. Inicialmente, esse banco de dados está vazio e não contém clientes. Os clientes chegarão um a um, e cada um deles é imediatamente adicionado ao banco de dados no momento da chegada. O j -ésimo cliente está interessado em receber tiberium em qualquer dia até o dia E_j inclusive. Para cada dia em que receber o tiberium, seu setor gerará R_j dólares de receita bruta. Assim, se a Kruskal combinar o fornecedor i com o cliente j , o lucro final de toda essa operação após a dedução do custo do tiberium será $(R_j - P_i) \times (E_j - S_i + 1)$, onde

$S_i \leq E_j$, pois, caso contrário, nenhum tiberio poderia ser fornecido.

A qualquer momento, o sistema da Kruskal pode calcular rapidamente, para qualquer fornecedor específico i , o cliente ideal entre os que estão no banco de dados, de modo que o lucro da operação ao combinar o fornecedor e o cliente seja maximizado, e pode informar esse lucro máximo. Pode ser que um lucro positivo para um fornecedor não possa ser obtido com nenhum dos clientes disponíveis; nesse caso, o sistema informa um lucro zero.

Observe que quando o sistema da Kruskal é solicitado a calcular o lucro máximo para um determinado fornecedor, esse fornecedor é combinado com no máximo um dos clientes disponíveis e, nesse caso, essa combinação não tem efeito algum sobre as operações futuras. Isso significa que tanto o fornecedor quanto o cliente podem ser considerados novamente para futuras combinações.

Sua tarefa é implementar o sistema de Kruskal.

Entrada

A primeira linha contém um número inteiro N ($1 \leq N \leq 2 \times 10^5$) indicando o número de fornecedores. A i -ésima das próximas N linhas descreve o fornecedor i com dois inteiros S_i e P_i ($1 \leq S_i, P_i \leq 10^9$), denotando respectivamente o dia de início e o preço por dia para o fornecedor. É garantido que $S_i < S_{i+1}$ e $P_i > P_{i+1}$ para $i = 1, 2, \dots, N - 1$.

A próxima linha contém um número inteiro Q ($1 \leq Q \leq 2 \times 10^5$) que representa o número de operações que devem ser processadas. As operações são descritas nas próximas Q linhas, na ordem em que são executadas no sistema, uma operação por linha. Há dois tipos de operações.

Se a operação adicionar um cliente ao banco de dados, a linha conterá a letra minúscula "c", seguido de dois números inteiros E e R ($1 \leq E, R \leq 10^9$), indicando respectivamente o dia final e a receita bruta por dia para o cliente.

Se a operação solicitar o cálculo do lucro máximo de um fornecedor, a linha conterá a letra minúscula "s", seguida de um número inteiro I ($1 \leq I \leq N$) que identifica o fornecedor. É garantido que a entrada contenha pelo menos uma operação desse tipo.

Saída

Emite uma linha para cada operação do tipo "s". A linha deve conter um número inteiro indicando o lucro máximo possível ao combinar um cliente disponível com um determinado fornecedor. Escreva os resultados das operações na ordem em que aparecem na entrada.

Entrada de amostra 1	Saída de amostra 1
4	0
2 8	18
4 5	35
7 3	28
9 2	16
11	84
s 1	16
c 10 10	28
s 1	108
s 2	
s 3	
s 4	
c 7 26	
s 2	
s 4	
s 3	
s 1	

Problema B - Jogo do quadro negro

Autor : Arthur Nascimento, Brasil

Carlinhos e Equalizer estão jogando um jogo. O jogo começa com $3N$ elementos, que são números inteiros, escritos em um quadro negro. Em seguida, durante N rodadas, as duas etapas a seguir são repetidas.

1. Carlinhos, o primeiro jogador, seleciona um elemento não escolhido e o marca com um círculo vermelho.
2. Equalizer, o segundo jogador, escolhe dois elementos não escolhidos, marca um deles com um quadrado azul e apaga o outro da lousa.

No final dessas rodadas, o quadro negro contém N elementos marcados em vermelho e N elementos marcados em azul, sem movimentos restantes. O jogo termina com um vencedor claro: se a soma dos elementos marcados em vermelho for diferente da soma dos elementos marcados em azul, Carlinhos sai vitorioso; caso contrário, Equalizer leva a vitória.

A figura abaixo mostra o único resultado possível para a primeira amostra. Nesse caso, o Equalizer ganha com certeza, não importa como ele jogue, ambas as somas serão iguais a 25.



Carlinhos, sentindo que o jogo está desequilibrado, procura determinar se pode garantir a vitória quando os dois jogadores jogam de forma ideal. Você pode ajudá-lo nessa tarefa?

Entrada

A primeira linha contém um número inteiro N ($1 \leq N \leq 1000$).

A segunda linha contém $3N$ números inteiros $B_1, B_2, \dots, B_{3N}$ ($-10^5 \leq B_i \leq 10^5$ para $i = 1, 2, \dots, 3N$), representando os números inicialmente escritos na lousa.

Saída

Emita uma única linha com a letra maiúscula "Y" se Carlinhos puder ganhar o jogo e a letra maiúscula "N" caso contrário, supondo que ambos os jogadores joguem de forma otimizada.

Entrada de amostra 1 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5	Saída de amostra 1 N
Entrada de amostra 2 2 1 2 4 8 16 32	Exemplo de saída 2 Y

Explicação da amostra 2:

Carlinhos vence independentemente da forma como joga, pois todos os subconjuntos têm somas distintas.

Entrada de amostra 3	Exemplo de saída 3
1 2 3 3	Y

Explicação da amostra 3:

Carlinhos pode ganhar se escolher o número 2. Observe que ele teria perdido se tivesse escolhido o 3.

Problema C -Candy Rush

Autor : Rafael Grandsire & Pedro Paniago, Brasil

É hora do rush! Você chegou ao fim do dia de trabalho e precisa comprar doces para todos os membros da sua família antes que o shopping feche.

Exclusividade e uniformidade são características muito valorizadas por sua família e, para impressioná-la, você elaborou um plano. Todos os doces dados a cada membro da família devem ser de uma única marca, e nenhum outro membro da família deve receber doces dessa mesma marca. Além disso, você não quer admitir que gosta mais de uns do que de outros, portanto, quer que todos recebam o mesmo número de doces.

O shopping tem uma loja que vende doces de K marcas diferentes. Coincidentemente, sua família é composta por exatamente K membros. Isso pode parecer muito fácil, mas é claro que há uma pegadinha.

A loja exibe seus doces alinhados em uma única prateleira. Você não tem tempo para selecionar os doces individualmente; em vez disso, deseja comprar um grupo de doces *contíguos* para concluir sua tarefa com eficiência. Isso significa que, ao comprar qualquer par de doces, você também deve comprar todos os doces localizados entre eles na prateleira.

Qual é o número máximo de doces que você pode comprar?

Entrada

A primeira linha contém dois números inteiros N e K ($1 \leq N, K \leq 4 \times 10^5$), indicando respectivamente o número de doces na prateleira e o número de membros da família. As marcas de doces são identificadas por números inteiros distintos de 1 a K .

A segunda linha contém N números inteiros $C_1, C_2, \dots, C_N$ ($1 \leq C_i \leq K$ para $i = 1, 2, \dots, N$), indicando a marca de cada doce na prateleira, na ordem da esquerda para a direita.

Saída

Envie uma única linha com um número inteiro indicando o número máximo de doces que você pode comprar para sua família. Lembre-se de que nenhum membro da família pode receber duas marcas de doces diferentes e nenhuma marca de doce pode ser comprada para dois membros diferentes da família. Além disso, cada membro da família deve receber o mesmo número de doces, e os doces devem ser comprados em um bloco contíguo na prateleira.

Entrada de amostra 1	Saída de amostra 1
6 2 2 2 1 1 2 2	4

Explicação da amostra 1:

Se você comprar do primeiro ao quarto bombom ou do terceiro ao sexto, poderá comprar dois bombons de cada marca.

Entrada de amostra 2	Exemplo de saída 2
7 3 2 1 2 1 2 2 3	0

Explicação da amostra 2:

Nenhum bloco contíguo de doces tem o mesmo número de doces para as marcas 1, 2 e 3.

Entrada de amostra 3	Exemplo de saída 3
3 4 3 4 2	0

Explicação da amostra 3:

Embora a loja seja conhecida por vender doces de K marcas diferentes, algumas marcas podem estar fora de estoque.

Problema D - Decifrando o WordWhiz

Autor : Paulo Cezar Pereira Costa, Brasil

O WordWhiz é um popular jogo de quebra-cabeça de palavras que desafia os jogadores a adivinhar uma palavra secreta em um número limitado de tentativas. O jogo usa um dicionário com N palavras. Cada palavra desse dicionário consiste em cinco letras minúsculas distintas.

O jogo começa com a apresentação ao jogador de uma grade vazia, que consiste em um número de linhas. Cada linha permite um único palpite. A tarefa do jogador é preencher as linhas com palavras contidas no dicionário até que a palavra secreta seja encontrada ou o jogador tenha usado todas as linhas disponíveis.

Depois que o jogador envia um palpite, o jogo fornece feedback colorindo as células onde o palpite foi escrito. O feedback consiste em três cores:

- Cinza ("X"): A letra na célula não faz parte da palavra secreta.
- Amarelo ("!"): A letra na célula faz parte da palavra secreta, mas está na posição errada.
- Verde ("*"): A letra na célula faz parte da palavra secreta e está na posição correta.

Para ilustrar, vamos considerar o cenário em que a palavra secreta é "hotel" e o jogador envia "blast" como seu palpite. Nesse caso, a primeira, a terceira e a quarta células ficariam cinza porque "b", "a" e "s" não estão presentes na palavra secreta "hotel". A segunda e a quinta células, entretanto, ficariam amarelas. Isso indica que "l" e "t" fazem parte da palavra secreta, mas aparecem em posições erradas: "l" deveria estar na quinta posição em vez de na segunda, enquanto "t" deveria estar na terceira posição em vez de na quinta. Esse feedback seria representado por "X!XX!".

Agora, se o jogador enviar "heart" (coração) como seu palpite, a terceira e a quarta células ainda ficarão cinza, pois "a" e "r" não estão em "hotel". A segunda e a quinta células ficariam novamente amarelas, porque mais uma vez "t" está na quinta posição (em vez da terceira) e, desta vez, "e" está na segunda posição, quando deveria estar na quarta. Entretanto, para esse palpite, a primeira célula ficaria verde, indicando que "h" é a primeira letra tanto no palpite "heart" (coração) quanto na palavra secreta "hotel". Esse feedback seria representado por "*!XX!".

Por fim, se o jogador enviar "hotel" como seu palpite, todas as células ficarão verdes, pois essa é a palavra secreta. Esse feedback seria representado por "*****".

Os feedbacks acima podem ser vistos na imagem a seguir.



Há algum tempo, sua empresa adicionou um player WordWhiz em seu site e agora deseja aprimorar o jogo adicionando a funcionalidade de exibir sessões de jogos anteriores. Entretanto, apenas o feedback de cada palpite foi armazenado, não as palavras enviadas. Isso significa que talvez não seja possível recuperar com precisão os palpites enviados em cada sessão e, antes de investir mais esforços, a empresa deseja analisar as sessões de jogo gravadas.

Dado um dicionário de palavras de cinco letras, a palavra secreta (incluída no dicionário) e o feedback de uma sessão de jogo, sua tarefa é determinar quantas palavras do dicionário poderiam ter sido enviadas como cada palpite.

Entrada

A primeira linha contém um número inteiro N ($1 \leq N \leq 1000$) indicando o número de palavras no dicionário.

Cada uma das próximas N linhas contém uma string que representa uma palavra do dicionário. Todas as cadeias de caracteres são diferentes e cada uma delas consiste em cinco letras minúsculas diferentes. A primeira cadeia é a palavra secreta da sessão do jogo.

A próxima linha contém um número inteiro G ($1 \leq G \leq 10$) indicando o número de palpites durante a sessão do jogo.

Cada uma das próximas linhas G contém uma cadeia de cinco caracteres que representa o feedback de um palpite. A string de feedback contém apenas os caracteres "X", "!" e "**", indicando respectivamente as cores cinza, amarela e verde.

É garantido que a entrada descreve uma sessão de jogo válida.

Saída

Saída de linhas G , de modo que a i -ésima contenha um número inteiro indicando quantas palavras do dicionário poderiam ter sido enviadas na i -ésima tentativa.

Entrada de amostra 1	Saída de amostra 1
6	1
hotel	1
cansado	1
coração	
explosão	
piloto	
vago	
3	
X!XX!	
*!XX!	

Explicação da amostra 1:

A única possibilidade é que o jogador tenha enviado palpites conforme descrito na declaração.

Entrada de amostra 2	Exemplo de saída 2
3	2
escala	2
tabela	2
bordo	2
5	2
X!X**	
X!X**	
X!X**	
X!X**	
X!X**	

Explicação da amostra 2:

O feedback quando "table" (mesa) ou "maple" (bordo) é enviado como palpite é "X!X**" (porque a palavra secreta é "scale" (escala)). Isso significa que, nessa sessão de jogo, o jogador poderia ter enviado qualquer uma dessas palavras em cada tentativa.

Entrada de amostra 3 4 escala tabela bordo sorriso 4 X!X** *XX** X!X** *****	Exemplo de saída 3 2 1 2 1
Entrada de amostra 4 5 latino mrica pensar resolver depurar 1 *****	Exemplo de saída 4 1

Problema E - Lucros elevados

Autor : Arthur Nascimento, Brasil

Marina, uma influenciadora digital que adora viajar pelo mundo, está embarcando em uma turnê promocional para uma marca de roupas femininas chamada $W^2 M$ (From Woman to Woman Marina). A jornada de Marina a leva por N cidades na América Latina, cada uma com seu charme único e identificada com um número inteiro distinto de 1 a N , chamado de índice de popularidade.

Para facilitar as viagens de Marina, a $W^2 M$ forneceu a ela $N - 1$ transferências, conectando pares de cidades de forma a garantir a acessibilidade a todas as N cidades. Marina pode passar por essas conexões quantas vezes quiser.

A missão de Marina é mostrar os vestidos da marca em cada uma das N cidades, com um detalhe. Cada vez que visita uma cidade pela primeira vez, ela deve escolher um vestido que nunca usou antes e capturar a essência da cidade em uma publicação na mídia social. Cada nova foto que ela compartilha atrai seguidores, criando expectativa para a próxima. O valor de expectativa para sua primeira foto é 1 e aumenta em 1 para cada foto subsequente.

Marina pode visitar qualquer cidade quantas vezes quiser, mas uma nova foto só deve ser publicada em sua primeira visita a uma cidade. Seu objetivo é maximizar o lucro de seu passeio, que é calculado como a soma do valor de antecipação de cada foto multiplicado pelo índice de popularidade da cidade onde a foto foi tirada. Mais precisamente, deixe p_i ser o índice de popularidade da cidade onde a i -ésima foto foi tirada. Com essas informações, o lucro pode ser calculado como

$$\sum_{i=1}^N i \times p_i = 1 \times p_1 + 2 \times p_2 + \dots + N \times p_N.$$

Agora, Marina precisa de sua ajuda. Considerando que a excursão deve começar na cidade $p_1 = R$, sua tarefa é ajudar Marina a determinar o lucro máximo que ela pode obter planejando estrategicamente a ordem das visitas às cidades.

Entrada

A primeira linha contém dois números inteiros N ($1 \leq N \leq 3 \times 10^5$) e R ($1 \leq R \leq N$), indicando, respectivamente, o número de cidades e a cidade inicial do tour.

Cada uma das próximas $N - 1$ linhas contém dois inteiros U e V ($1 \leq U, V \leq N$ e $U \neq V$), indicando que há uma transferência entre as cidades U e V . É garantido que é possível chegar a todas as cidades usando as transferências.

Saída

Produza uma única linha com um número inteiro indicando o lucro máximo que Marina pode obter em sua turnê promocional.

Entrada de amostra 1 7 3 3 5 3 7 5 1 5 4 7 2 7 6	Saída de amostra 1 121
Entrada de amostra 2 1 1	Exemplo de saída 2 1

Problema F - Para frente e para trás

Autor : Alejandro Strejilevich de Loma, Argentina

Um sistema planetário distante tem um único sol e $N - 1$ planetas. Cada planeta é identificado por um número inteiro distinto de 2 a N . No planeta b , os números são representados usando a base b .

Um número palindrômico é um número que permanece o mesmo quando seus dígitos são escritos tanto para frente quanto para trás. Nesse contexto, os zeros à esquerda não são considerados ao determinar se um número é palindrômico.

O mesmo número pode ser palindrômico em uma base planetária, mas não em outra. Por exemplo, na base 10, o número 33 é palindrômico. Ele também é palindrômico na base 2 e na base 32, mas não em bases como 3 ou 33, pois $33_{10} = 100001_2 = 1020_3 = 11_{32} = 10_{.33}$.

Os habitantes desse sistema planetário têm um gosto peculiar por números palindrômicos e querem saber quais planetas tornam o número N um número palindrômico quando representado em sua base. Sua tarefa é ajudá-los com esse desafio cósmico.

Entrada

A entrada consiste em uma única linha que contém um número inteiro N ($2 \leq N \leq 10^{12}$) indicando o número a ser verificado quanto à representação palindrômica. N é fornecido na base 10.

Saída

Produza uma única linha com uma lista crescente de números inteiros no intervalo $[2, N]$, indicando os planetas nos quais N é um número palindrômico quando expresso na base do identificador do planeta. Produza esses números inteiros na base 10. Se N não for palindrômico em nenhum dos planetas, produza o caractere "*" (asterisco).

Entrada de amostra 1	Saída de amostra 1
33	2 10 32
Entrada de amostra 2	Exemplo de saída 2
3	2
Entrada de amostra 3	Exemplo de saída 3
2	*

Problema G -GPS em uma Terra plana

Autor : Mário da Silva, Brasil

No dia em que os alienígenas finalmente atacaram a humanidade, ninguém poderia prever a arma escolhida por eles. Nada de armas nucleares, meteoros, lasers ou monstros gigantes. Em vez disso, nosso planeta foi subjugado com o poder da física!

Especificamente, os alienígenas transformaram a Terra em uma superfície plana e bidimensional, prejudicando para sempre nossa capacidade de navegar no espaço. Embora frustrada, a humanidade sobreviveu e retomamos nossas vidas da melhor forma possível. Essa nova existência bidimensional exige muitos ajustes, inclusive o uso do GPS (Sistema de Posicionamento Global).

O GPS normalmente funciona usando ondas de rádio para medir as distâncias euclidianas do usuário a vários pontos de referência (satélites) e usando essas distâncias para calcular as coordenadas do usuário. Entretanto, a Terra, agora plana, tem duas peculiaridades às quais precisamos nos adaptar:

- Sem satélites em órbita, precisamos usar torres de rádio. Cada torre de rádio agora tem cobertura sobre todo o planeta devido à superfície plana.
- As ondas de rádio, que se propagam de forma diferente em um mundo bidimensional, exigem uma mudança da distância euclidiana para a distância de Manhattan para cálculos precisos. Considerando dois pontos quaisquer (X_1, Y_1) e (X_2, Y_2) , a distância de Manhattan entre eles é definida como $|X_1 - X_2| + |Y_1 - Y_2|$.

Sua tarefa é escrever um software para esses cálculos de GPS adaptados. Dada uma lista de localizações de N torres de rádio de referência e suas respectivas distâncias de Manhattan para o usuário do GPS, seu algoritmo deve fornecer uma lista de possíveis localizações do usuário. Essas possíveis localizações do usuário são limitadas àquelas que estão exatamente na distância de Manhattan medida de cada torre de rádio de referência. O GPS ainda está na fase inicial de testes, portanto, a localização real do usuário está limitada a coordenadas inteiras.

Entrada

A primeira linha contém um número inteiro N ($1 \leq N \leq 10^5$) indicando o número de torres de rádio de referência.

Cada uma das próximas N linhas descreve uma torre com três números inteiros X, Y ($-10^4 \leq X, Y \leq 10^4$) e D ($0 \leq D \leq 4 \times 10^4$), representando que uma torre com coordenadas (X, Y) está a uma distância D de Manhattan do usuário do GPS. Não há duas torres com a mesma localização. É garantido que os dados de entrada são confiáveis, apontando um conjunto finito e não vazio de possíveis localizações para um usuário com coordenadas inteiras.

Saída

Saída de várias linhas. Cada linha deve conter um par diferente de inteiros X_u e Y_u indicando que (X_u, Y_u) é um local de usuário compatível com os dados de entrada. As linhas devem ser classificadas pelo valor não decrescente de X_u , desempatando pelo valor crescente de Y_u .

Entrada de amostra 1 2 1 1 5 7 0 4	Saída de amostra 1 4 -1 5 2
Entrada de amostra 2 2 1 1 5 5 5 3	Exemplo de saída 2 2 5 3 4 4 3 5 2

Problema H -Saúde em perigo

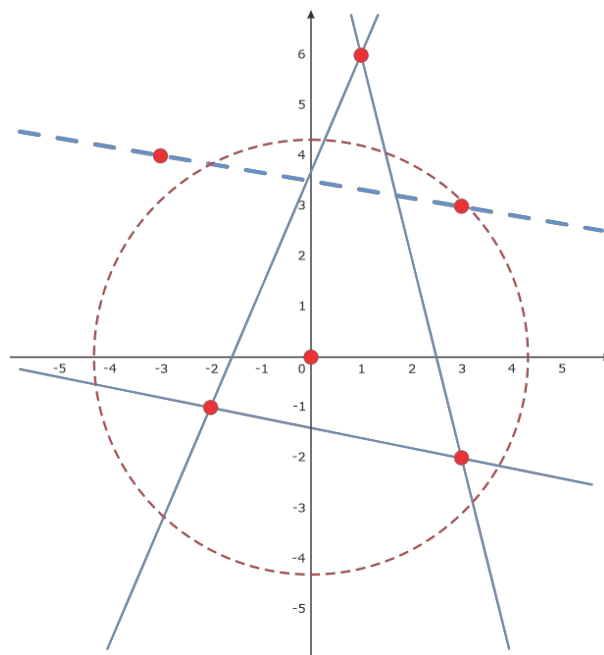
Autor : Victor de Sousa Lamarca, Brasil

Na selva congelada, um urso polar faz sua casa em uma vasta geleira, representada como um plano 2D. As coordenadas da toca do urso são $(0, 0)$. Para ser saudável, toda vez que o urso acorda em sua toca, ele caminha até qualquer ponto a uma distância exatamente D da toca (medida usando a distância euclidiana).

Com o urso enfrentando os desafios de um clima em mudança, uma equipe de cientistas e matemáticos dedicados se propôs a ajudar. Eles receberam um relatório detalhado repleto de previsões do impacto iminente do aquecimento global sobre a geleira nos próximos dias. O relatório contém as previsões em ordem cronológica, e cada uma delas é representada por uma linha infinita que corresponde a um evento de derretimento. Depois que cada previsão se torna realidade, a linha que a representa não pode mais ser cruzada pelo urso.

Inicialmente, considera-se que a geleira é infinitamente grande em todas as direções, e o urso pode andar livremente. No entanto, como equipe de cientistas e matemáticos, vocês entendem o dilema do urso: de acordo com as previsões, a geleira pode acabar encolhendo de tal forma que o urso não poderá mais ficar saudável. Sua tarefa é calcular o primeiro momento em que isso ocorrerá, ou seja, quando o urso não puder alcançar nenhum ponto a uma distância exatamente D da toca.

A figura abaixo mostra a primeira amostra. A circunferência contém os pontos a uma distância exatamente D da toca. Quando apenas as três primeiras previsões são consideradas (linhas sólidas), o urso ainda pode alcançar pontos na circunferência. Quando a quarta previsão (linha tracejada) também é considerada, nenhum ponto da circunferência pode ser alcançado a partir da toca.



Entrada

A primeira linha contém um número inteiro N ($1 \leq N \leq 2 \times 10^5$) e um número racional D com no máximo cinco dígitos após o ponto decimal ($1 \leq D \leq 10^6$). O valor N indica o número de previsões, enquanto D representa a distância da cova.

Cada uma das próximas N linhas descreve uma previsão com quatro números inteiros X_1, Y_1, X_2, Y_2 ($-10^6 \leq X_1, Y_1, X_2, Y_2 \leq 10^6$ e $(X_1, Y_1) \neq (X_2, Y_2)$), que definem uma linha infinita no plano que passa por (X_1, Y_1) e (X_2, Y_2) . Cada previsão indica que a linha correspondente não pode mais ser cruzada pelo urso. As previsões são fornecidas em ordem cronológica e são identificadas por números inteiros distintos de 1 a N , de acordo com essa ordem. É garantido que nenhuma previsão defina uma linha que passe pela toca.

Saída

Produza uma única linha com um número inteiro identificando a primeira previsão que indica que nenhum ponto a uma distância exatamente D da toca pode ser alcançado pelo urso. Se essa situação nunca ocorrer, produza o caractere "*" (asterisco) em seu lugar. É garantido que as variações no valor de D em um intervalo de $\pm 10^{-5}$ do valor fornecido na entrada não alteram a saída.

Entrada de amostra 1 5 4.321 -2 -1 3 -2 1 6 3 -2 1 6 -2 -1 -3 4 3 3 -2 1 5 4	Saída de amostra 1 4
Entrada de amostra 2 5 2 1 0 1 1 -1 0 -1 -1 3 1 1 3 1 3 3 1 0 4 4 0	Exemplo de saída 2 *

Problema I - Inversões

Autor : Gabriel Poesia, Brasil

Todos os anos, matemáticos e cientistas da computação de todo o mundo se reúnem para o prestigiado Concurso de Quebra-cabeça de Contagem de Inversões (ICPC). Para o próximo ICPC, os organizadores prepararam o seguinte desafio: dada uma string S composta de letras minúsculas, conte o número de *inversões* nela. Uma inversão é um par de índices $i < j$, de modo que S_i (a letra na posição i) vem depois de S_j no alfabeto.

No entanto, no mês passado, um grupo de pesquisadores excepcionais desenvolveu um algoritmo sofisticado que pode contar as inversões em uma cadeia de caracteres com extrema rapidez. Embora essa tenha sido uma ótima notícia para o avanço da ciência, foi um pesadelo para a equipe do ICPC, pois o desafio planejado agora está obsoleto.

Essa questão foi levada ao criador de problemas principal, que apresentou uma ideia inteligente. Em vez de simplesmente receber uma sequência S , eles deveriam pedir aos participantes que repetissem essa sequência N vezes antes de contar as inversões. Se os juízes definirem N como suficientemente grande, em algum momento o algoritmo proposto pelos pesquisadores começará a ser muito lento. Satisfeita com essa ideia, a equipe do ICPC prosseguiu com a organização do próximo concurso.

Infelizmente, agora os juízes não sabem mais as respostas para os arquivos de entrada e, portanto, não podem julgar os envios! O ICPC acabou de ser iniciado e os participantes estão prestes a começar a enviar suas soluções. Por favor, ajude os juízes computando as respostas, para que o ICPC possa ser realizado sem problemas.

Entrada

A primeira linha contém uma string S ($1 \leq |S| \leq 10^5$), que é composta de letras minúsculas.

A segunda linha contém um número inteiro N ($1 \leq N \leq 10^{12}$) que indica quantas vezes a string S deve ser repetida.

Saída

Produza uma única linha com um número inteiro indicando o número de inversões na string S^N (S repetido N vezes). Como esse número pode ser muito grande, gere o restante da divisão por $10^9 + 7$.

Entrada de amostra 1 ba 1	Saída de amostra 1 1
Entrada de amostra 2 ab 3	Exemplo de saída 2 3
Entrada de amostra 3 zkba 1	Exemplo de saída 3 6
Entrada de amostra 4 cab ine 7	Exemplo de saída 4 77

Problema J -Jornada do Ladrão

Autor : Giovanna Kobus Conrado, Brasil

Monopolis é um país bonito e rico. Entre suas características impressionantes está o layout do país, onde N cidades são interconectadas por $N - 1$ estradas de igual comprimento, permitindo viajar entre quaisquer duas cidades.

Outro recurso exclusivo do Monopolis é que cada cidade tem um único banco com uma quantidade diferente de dinheiro. Assim, é possível atribuir um número distinto de 1 a N a cada cidade, representando sua classificação de riqueza em relação às outras cidades, com a cidade 1 tendo menos dinheiro e a cidade N tendo mais.

Rob está planejando uma "viagem de negócios" a Monópolis. O negócio de Rob é, de fato, roubar. Roubar bancos, para ser mais preciso. Rob é um assaltante ambicioso e segue um modus operandi específico: ele só visa bancos com mais dinheiro do que aquele que acabou de roubar. Assim, depois de roubar em uma cidade i , ele vai para a cidade mais próxima que tenha mais dinheiro. Se houver várias cidades com mais dinheiro do que i na mesma distância, ele seleciona a que tem menos dinheiro. Se não houver nenhuma cidade mais rica do que i , ele permanecerá na mesma cidade e refletirá sobre suas ações.

Embora Rob esteja bem definido em seu modus operandi, ele ainda está planejando sua viagem de negócios a Monópolis e, então, pede sua ajuda. Rob quer saber, para cada cidade i , qual seria a próxima cidade a ser visitada, caso seu primeiro assalto seja na cidade i .

Entrada

A primeira linha contém um número inteiro N ($1 \leq N \leq 10^5$) que representa o número de cidades em Monópolis. Cada cidade é identificada por um número inteiro distinto de 1 a N , em ordem crescente de riqueza.

Cada uma das próximas $N - 1$ linhas contém dois números inteiros U e V ($1 \leq U, V \leq N$ e $U \neq V$), indicando que há uma estrada de mão dupla entre as cidades U e V . É garantido que há um caminho entre cada par de cidades usando as estradas fornecidas.

Saída

Produza uma única linha com N números inteiros, de modo que o i -ésimo deles indique a próxima cidade que Rob visitaria caso seu primeiro roubo fosse na cidade i .

Entrada de amostra 1 6 1 6 2 5 4 5 3 5 5 6	Saída de amostra 1 6 5 5 5 6 6
Entrada de amostra 2 5 5 1 1 3 3 2 2 4	Exemplo de saída 2 3 3 4 5 5
Entrada de amostra 3 1	Exemplo de saída 3 1

Problema K -Keen on Order

Autor : Arthur Nascimento, Brasil

A Nloglonia está organizando um festival de cinema que se estende por N dias. Há K filmes diferentes e, em cada um dos N dias, um deles será exibido. Cada um dos filmes disponíveis pode ser exibido em vários dias ou não ser exibido durante o festival.

A programação do festival é apresentada como uma matriz de números inteiros V de tamanho N , com $1 \leq V_i \leq K$, indicando qual filme será exibido em cada dia. Bob quer assistir a todos os K filmes e acredita firmemente que a ordem em que os assistirá afetará significativamente sua experiência. Então, agora ele se pergunta: é verdade que, para cada ordem dos filmes, ele poderia escolher K dias para visitar o festival e assistir a esses filmes nessa ordem? Em termos mais formais, é verdade que toda permutação de $1, 2, \dots, K$ é uma subsequência de V ? Se esse não for o caso, Bob também quer que você encontre alguma permutação (qualquer uma) que não seja.

Entrada

A primeira linha contém dois números inteiros N e K ($1 \leq N, K \leq 300$), indicando, respectivamente, o número de dias de duração do festival de cinema e o número de filmes disponíveis para exibição.

A segunda linha contém N números inteiros $V_1, V_2, \dots, V_N$ ($1 \leq V_i \leq K$ para $i = 1, 2, \dots, N$), indicando que o filme V_i será exibido no dia i .

Saída

Produza uma única linha com K inteiros mostrando uma permutação de $1, 2, \dots, K$ que não seja uma subsequência de V . Se toda permutação for uma subsequência de V , em vez disso, produza o caractere "*" (asterisco).

Entrada de amostra 1 9 3 1 2 3 1 2 3 1 2 3	Saída de amostra 1 *
Entrada de amostra 2 11 4 1 2 3 4 2 3 3 2 4 1 4	Exemplo de saída 2 3 4 1 2
Entrada de amostra 3 11 4 1 2 3 4 2 3 3 2 4 1 4	Exemplo de saída 3 4 1 2 3
Entrada de amostra 4 5 6 6 5 4 3 2	Exemplo de saída 4 6 5 4 3 2 1

Problema L - Latam++

Autor : Agust'in Santiago Gutierrez, Argentina

O mundo ainda não tem linguagens de programação suficientes. Para ajudar com isso, o ICPC (Internal Committee for the Perfection of C) está planejando criar uma nova linguagem de programação: Latam++.

No Latam++, um nome de variável consiste exclusivamente em uma ou mais letras minúsculas do alfabeto

Alfabeto inglês. Uma expressão válida é uma cadeia de caracteres "bem formada", que expressa como combinar variáveis usando os quatro operadores aritméticos binários "+", "-", "*" e "/", possivelmente com parênteses.

Formalmente, as expressões válidas são exatamente as cadeias de caracteres que podem ser produzidas pelas regras a seguir.

- Um nome de variável é uma expressão válida.
- O uso de parênteses ao redor de qualquer expressão válida produz outra expressão válida.
- Se A e B forem expressões válidas, então a concatenação AcB é uma expressão válida, onde c é qualquer um dos quatro operadores aritméticos binários "+", "-", "*" e "/".

Portanto, todas as expressões a seguir são válidas:

- $a+b$
- $a+b*(c+b)$
- $atoms+boots*(charly+bob)$
- $((a))^*(bbasdsaewe/a/a/a)$

Por outro lado, as expressões a seguir não são válidas:

- $a+$
- $a+b(c+b)$
- $atoms+boots*((charly+bob)$
- $((()))*(bbasdsaewe/a/a/a)$

A linguagem está longe de estar completa, e provavelmente serão necessárias décadas de debates no ICPC até que a primeira versão do Latam++ seja lançada. Enquanto isso, vamos nos concentrar apenas em um recurso específico e muito especial de seu compilador, chamado Automatic Valid Substring Expression Counting (AVSEC).

O AVSEC é um recurso extremamente útil, no qual o compilador informa o número total de substrings de uma determinada string que são expressões válidas. Sua tarefa é implementar o AVSEC.

Para fins de contagem, duas substrings são consideradas diferentes se começarem ou terminarem em índices diferentes, mesmo que as strings correspondentes sejam idênticas (ou seja, são a mesma sequência de caracteres).

Entrada

A entrada consiste em uma única linha que contém uma string S ($1 \leq |S| \leq 2 \times 10^5$), que é composta de letras minúsculas, parênteses de abertura ou fechamento e os quatro caracteres "+", "-", "*" e "/".

Saída

Emite uma única linha com um número inteiro indicando o número de substrings de S que são expressões válidas.

Entrada de amostra 1	Saída de amostra 1
a+b(c+b)	7

Explicação da amostra 1:

As sete substrings são "a", "a+b", "b" (terceiro caractere), "(c+b)", "c", "c+b" e "b" (sétimo caractere).

Entrada de amostra 2	Exemplo de saída 2
aa	3

Entrada de amostra 3	Exemplo de saída 3
a-a	3

Problema M - Ponto de encontro

Autor : Daniel Bossle, Brasil

Seu amigo Pedro sempre fica muito animado com as atividades em grupo. Em sua empolgação, ele corre tão rápido para o ponto de encontro que se cansa antes de chegar. Um dia, você decidiu coletar dados sobre esse fenômeno e, surpreendentemente, percebeu que ele sempre se cansa exatamente no ponto médio do percurso. Em outras palavras, ele se cansa quando já percorreu metade da distância que pretendia percorrer.

Sua cidade tem N cruzamentos identificados por números inteiros distintos de 1 a N e M vias de mão dupla. Cada estrada tem um comprimento e conecta um par específico de cruzamentos, de modo que há um caminho na cidade entre cada par de cruzamentos. A distância entre dois cruzamentos é o comprimento de um caminho mínimo entre esses cruzamentos.

Pedro mora no cruzamento P , e seu grupo de amigos decidiu se encontrar no cruzamento G ainda hoje. Depois de pensar um pouco, você elaborou o seguinte plano para que Pedro chegue a tempo. Você informará a ele um ponto de encontro enganoso para que ele se canse exatamente em G . Para que esse plano funcione, a encruzilhada G deve pertencer a todos os caminhos que Pedro poderia tomar ao ir de P até o ponto de encontro enganoso e, para cada um desses caminhos, Pedro deve se cansar exatamente em G . Felizmente, você sabe que Pedro é um bom planejador e nunca tomaria um caminho mais longo do que o necessário.

Agora você se pergunta: qual cruzamento funcionaria como esse ponto de encontro enganoso?

Entrada

A primeira linha contém dois números inteiros N ($2 \leq N \leq 10^5$) e M ($1 \leq M \leq 10^5$), indicando respectivamente o número de cruzamentos e o número de estradas na cidade. Cada cruzamento é identificado por um número inteiro distinto de 1 a N .

A segunda linha contém dois números inteiros P e G ($1 \leq P, G \leq N$ e $P \neq G$), indicando respectivamente o cruzamento onde Pedro mora e o ponto de encontro correto.

Cada uma das próximas M linhas descreve uma estrada com três números inteiros U , V e D ($1 \leq U, V \leq N$, $U \neq V$ e $1 \leq D \leq 10^9$), representando que há uma estrada de mão dupla de comprimento D entre os cruzamentos U e V .

É garantido que há um caminho entre cada par de cruzamentos usando as estradas fornecidas, e há no máximo uma estrada entre cada par de cruzamentos.

Saída

Produza uma única linha com uma lista crescente de números inteiros indicando os cruzamentos que funcionariam para o seu plano. Se nenhum cruzamento funcionar, produza o caractere "*" (asterisco) em seu lugar.

Entrada de amostra 1	Saída de amostra 1
4 5 1 3 1 3 1 2 1 3 2 4 3 4 3 1 3 2 1	2 4

Entrada de amostra 2 4 5 1 3 1 3 1 2 1 2 2 4 3 4 3 1 3 2 1	Exemplo de saída 2 4
Entrada de amostra 3 3 2 1 2 1 2 100000 2 3 99999	Exemplo de saída 3 *
Entrada de amostra 4 4 4 4 3 3 4 1 4 1 1 1 2 1 2 3 1	Exemplo de saída 4 *